

ASTRA Production

Arquitectura autónoma de razonamiento y validación de teoremas científicos

Nelson Bolívar
Science-Stack Research Group

Versión de arquitectura: julio de 2026

Resumen

ASTRA Production (*Autonomous Symbolic Theorem Reasoning Architecture*) es un motor multiagente de investigación que transforma una intuición científica en una hipótesis falsable, la traduce a un experimento computacional ejecutable y somete el resultado a análisis crítico antes de aceptar una conclusión. Su principio central es separar la generación lingüística de la evidencia: los modelos de lenguaje proponen, formalizan y analizan; un oráculo de cálculo simbólico o numérico ejecuta el programa de validación y devuelve evidencia reproducible. El sistema admite ciclos aislados y campañas autónomas de investigación, varios proveedores de modelos, motores matemáticos locales o remotos, corrección automática de código, aprobación humana de teoremas e informes persistentes. Este documento describe la arquitectura implementada actualmente, sus garantías y sus límites.

1 Motivación y principio epistemológico

Los modelos de lenguaje son útiles para formular hipótesis, conectar dominios y escribir programas, pero una respuesta plausible no constituye una demostración. ASTRA adopta por ello una división explícita de responsabilidades:

- la **capa heurística** propone conjeturas, genera código y explica resultados;
- la **capa de ejecución** calcula con motores simbólicos, numéricos o lógicos reales;
- la **capa de control** conserva el estado, aplica reintentos, protege el veredicto y solicita decisiones humanas;
- la **capa documental** registra código, salida, análisis, proveedores y evolución de la investigación.

La afirmación de trabajo no es que ASTRA automatice la verdad matemática, sino que convierte el razonamiento asistido por IA en un proceso inspeccionable:

intuición $\longrightarrow$ hipótesis falsable $\longrightarrow$ programa
 $\longrightarrow$ evidencia ejecutada $\longrightarrow$ veredicto revisable.

La salida de un modelo nunca reemplaza la ejecución. A su vez, una ejecución exitosa no basta por sí sola: el programa debe representar correctamente la afirmación y producir una señal discriminante entre validación y refutación.

2 Arquitectura vigente

ASTRA se ejecuta como una aplicación local con interfaz web. El servidor Flask expone la interfaz y una API de control; un orquestador asíncrono mantiene la máquina de estados en segundo plano. Los agentes se comunican con proveedores LLM mediante una capa común, mientras que el oráculo enruta el código hacia el motor adecuado. La interfaz web, el servidor MCP y la herramienta por subproceso invocan el mismo ciclo canónico: no mantienen implementaciones científicas paralelas.

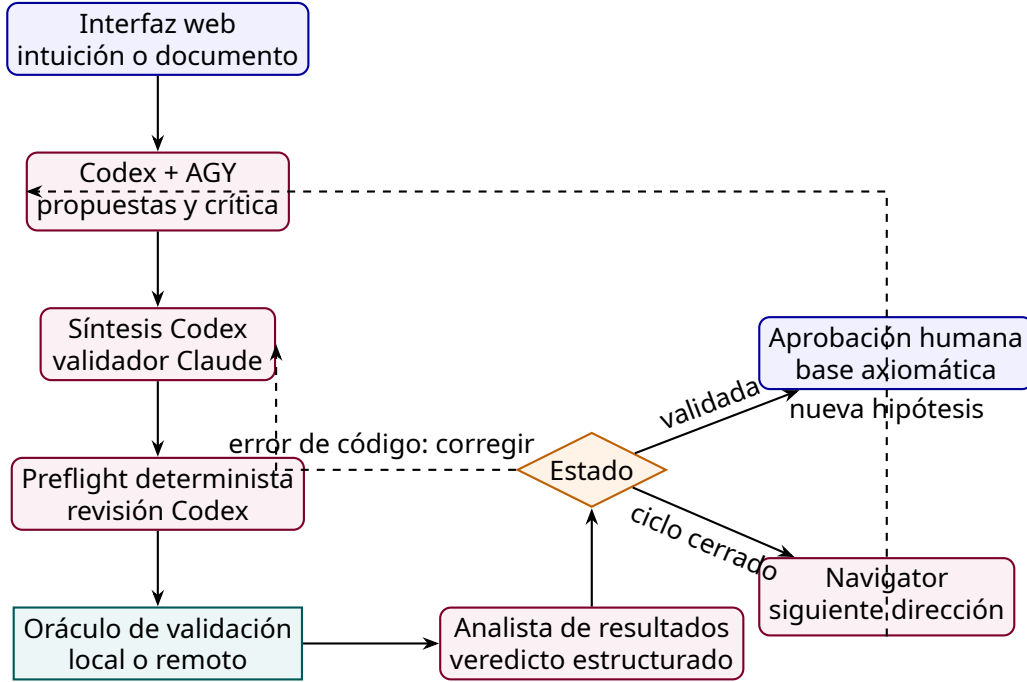


Figura 1: Flujo lógico de ASTRA Production. El Navigator interviene en el modo de investigación; la aprobación humana protege la incorporación de resultados a la base axiomática.

2.1 Entrada e interfaz

El usuario puede introducir una afirmación directamente o cargar material PDF/TXT. La aplicación ofrece dos modalidades:

- **Ciclo único:** evalúa una intuición concreta mediante una pasada completa.
- **Bucle de investigación:** descompone una pregunta macroscópica en una secuencia de conjeturas relacionadas y mantiene ramas alternativas.

La interfaz muestra el estado, la conjetura vigente, el script generado, la salida del oráculo, la base axiomática, los informes y el registro de eventos. También permite detener la ejecución, aprobar o rechazar resultados y configurar los proveedores por rol. La caché de ciclos incluye el objetivo, la base axiomática, el resumen de la trayectoria reflexiva, la distancia al último hito y la configuración no secreta de modelos y controles; por ello una dirección repetida en un contexto científico nuevo no reutiliza una navegación obsoleta.

2.2 Agente de conjeturas

Codex GPT-5.6 Sol y AGY reciben el mismo objetivo final, la dirección actual y el contexto acumulado. Proponen en paralelo, cada uno critica adversarialmente la propuesta rival y

Codex sintetiza una única conjetura falsable con variables, supuestos, dominio y criterio de éxito. En el modo iterativo también utilizan teoremas establecidos y refutaciones anteriores para evitar repetir rutas cerradas.

2.3 Traductor formal

El traductor convierte la conjetura en un script autocontenido. El código debe calcular una cantidad verificable: un residual simbólico, una identidad, condiciones iniciales, una desigualdad, un contraejemplo, una propiedad tensorial o una comparación numérica con tolerancias explícitas. Puede declarar el motor mediante una cabecera como `ASTRA_ENGINE` y sugerir ejecución local o remota con `ASTRA_ORACLE`.

El traductor no emite el veredicto científico. Construye el experimento que permitirá decidirlo.

2.4 Preflight y revisión independiente

Antes de ejecutar, controles deterministas buscan patrones que podrían producir una validación falsa, errores de importación y fallos operacionales conocidos. Codex compara luego el script de Claude con el objetivo compartido y la conjetura exacta. Si encuentra un defecto reparable, Claude recibe instrucciones acotadas y devuelve un parche de reemplazos exactos; Codex no pasa a ser autor del validador. Un rechazo del revisor o un parche no aplicable produce abstención o error de herramienta, nunca evidencia científica.

2.5 Oráculo de validación

El oráculo ejecuta el programa y captura código de retorno, salida estándar, errores y metadatos del motor. Python constituye la base común, con bibliotecas como SymPy, Z3, NumPy, SciPy, mpmath, EinsteinPy, Pint, fluids y QuTiP. ASTRA también puede enrutar trabajos a SageMath, Maxima o Cadabra cuando están disponibles. En Windows, `ASTRA_WSL_DISTRO` fija la distribución WSL2 deseada para que la ejecución no cambie silenciosamente si se modifica la distribución predeterminada de la máquina. La estación de producción utiliza Debian/WSL2 para SageMath, Maxima, Cadabra y el entorno local Lean/Mathlib.

La ejecución puede ser:

- **local**, adecuada para álgebra simbólica y cálculos moderados;
- **remota**, mediante un trabajador Linux por SSH para cargas pesadas, GPU o herramientas instaladas en la estación de cálculo.

La política puede fijarse por configuración o inferirse de marcadores y características del script. La ausencia de un motor opcional debe producir un fallo explícito, no una validación ficticia.

2.6 Analista y guardia de veredicto

El analista recibe la conjetura y la evidencia bruta. Devuelve una respuesta estructurada con uno de cuatro estados principales:

Estado	Significado
VALIDATED	La evidencia computacional respalda la hipótesis formulada.
REFUTED	La ejecución produce un contraejemplo o contradice la hipótesis.
CODE_ERROR	El programa no logró ejecutar correctamente la prueba.
API_ERROR	Falló el proveedor LLM, la red o la cuota correspondiente.

Cuando aparece `CODE_ERROR`, ASTRA solicita código corregido y repite la validación hasta el límite configurado. La guardia de veredicto restringe conclusiones incompatibles con la evidencia de ejecución. Esto reduce, aunque no elimina, el riesgo de que el analista convierta un fallo técnico en una afirmación científica.

2.7 Aprobación humana y base axiomática

Una hipótesis marcada como validada queda en espera. Sólo después de la aprobación del investigador se incorpora como teorema establecido. Las refutaciones, con su justificación, también alimentan el contexto para bloquear caminos ya descartados. Esta puerta humana reconoce que la corrección del programa, los supuestos físicos y el alcance de la conclusión todavía requieren juicio experto.

3 Bucle autónomo de investigación

En una campaña, ASTRA mantiene una *Research Session* asociada a una pregunta macroscópica. Después de cada ciclo, el Navigator evalúa el progreso y propone:

- la siguiente dirección en profundidad;
- ramas paralelas que conviene conservar;
- si se alcanzó un hito que requiere revisión;
- si la pregunta macroscópica puede considerarse resuelta.

Si una conjetura fue validada, el sistema puede generalizarla, explorar sus fronteras o aplicarla a otro régimen. Si fue refutada, busca una alternativa que evite la causa concreta del fracaso. El investigador puede continuar con la propuesta, redirigir el trabajo o activar una rama guardada. En modo autónomo, ASTRA atraviesa los hitos sin pausa hasta que el Navigator declara resolución, se alcanza el tiempo máximo o el usuario detiene la sesión.

Este procedimiento es profundidad-primero con memoria de ramas; no equivale a una búsqueda exhaustiva ni garantiza que la declaración de resolución sea correcta.

4 Control de ASTRA desde un agente

ASTRA no está limitado a su interfaz web. El servidor *Model Context Protocol* (MCP) expone el oráculo como herramientas que pueden invocar Codex, Claude Code, Gemini u otro cliente compatible. De este modo, el agente que conversa con el investigador puede formular el plan, escribir el programa de prueba, pedir la ejecución y examinar la evidencia sin copiar manualmente resultados entre aplicaciones.

La interfaz actual ofrece seis operaciones principales:

Herramienta	Función
<code>astra_execute</code>	Ejecuta un script ya escrito en el oráculo local, remoto o automático.
<code>astra_cycle</code>	Ejecuta el ciclo completo: conjetura, traducción, oráculo y análisis.
<code>astra_probe</code>	Consulta la fase y el <i>heartbeat</i> de un ciclo sin interrumpirlo.
<code>astra_submit</code>	Envía un cálculo largo como trabajo desacoplado.
<code>astra_job</code>	Lista o sondea trabajos, salida incremental y resultado final.
<code>astra_status</code>	Comprueba la disponibilidad del trabajador remoto.

El MCP se registra apuntando el cliente al intérprete del entorno de ASTRA y a `mcp_server/server.py`; después se abre una sesión nueva del agente para cargar las herramientas. El investigador puede entonces pedir, por ejemplo: “formula una prueba independiente del escalar de Ricci, ejecútala con `astra_execute` y no aceptes el resultado hasta revisar los residuales”. Para cálculos de más de unos minutos conviene usar `astra_submit` y seguirlos con `astra_job`.

Esta modalidad convierte al agente conversacional en la capa de planificación y a ASTRA en la capa de ejecución trazable. El agente no recibe autoridad adicional: sigue obligado a presentar código, supuestos, salida y criterio de decisión.

5 Extensión mediante motores y paquetes científicos

La capa del oráculo es extensible y no constituye una lista cerrada. Todo paquete disponible en el entorno donde corre el script puede participar en una validación. Entre los paquetes científicos desarrollados por el propio equipo se encuentran:

```
GR_python  pyWarpFactory  warp_bubble_optimization.
```

El requisito de integración es sencillo: el script debe poder importar el paquete, fijar entradas reproducibles, producir evidencia inspeccionable y terminar con un veredicto explícito. El mismo paquete debe instalarse en el entorno local o en el trabajador remoto elegido.

Esta capacidad permite reutilizar solvers, métricas, optimizadores, análisis de condiciones de energía y bancos de pruebas ya validados. Para disminuir errores comunes, una conclusión importante debería combinar patas independientes: por ejemplo, una identidad simbólica, evaluaciones numéricas con semilla fija y un límite conocido.

5.1 Lean 4 y Mathlib

Lean 4 se integra como motor opcional mediante la marca `# ASTRA_ENGINE: lean`. El traductor puede generar un archivo que importe Mathlib; ASTRA lo entrega a un proyecto nativo fijado, al proyecto fijado en Debian/WSL2 o a la ruta formal configurada en ASTRUM. La ruta local de producción fija Lean 4.30.0 y el commit de Mathlib que comienza por `c5ea00351c28`. La aceptación por el kernel de Lean proporciona verificación mecánica apropiada para lógica formal, matemáticas discretas, álgebra y teoremas que ya puedan expresarse en el lenguaje del asistente de pruebas.

Lean no sustituye los cálculos físicos numéricos ni prueba que la formalización represente correctamente al sistema real. Su garantía se aplica al enunciado formal y a los axiomas importados. Si el compilador rechaza la prueba, ASTRA lo clasifica como error de código y puede solicitar una corrección; nunca debe convertir el rechazo en una refutación del teorema.

5.2 Wolfram Mathematica como oráculo complementario

Cuando el modelo dispone del complemento de Wolfram o de Mathematica Agent Bridge, el agente puede escribir y ejecutar Wolfram Language directamente. El bridge local comunica procesos externos con un kernel de Mathematica mediante una cola JSON y puede operar sobre un notebook abierto o en modo *headless* con `wolframscript`. Entre sus acciones se encuentran:

```
evaluate-wl, simplify-symbolic, compare-expressions, commutator y  
gr-curvature.
```

El bridge también permite leer, escribir y evaluar celdas.

Mathematica puede actuar como segundo validador independiente: el agente ejecuta una prueba en ASTRA y repite la identidad, el tensor o el límite con el bridge. La concordancia entre motores de implementación distinta fortalece la evidencia; una discrepancia obliga a revisar convenciones, dominios y supuestos. El bridge evalúa código con los privilegios del usuario local, por lo que sólo debe habilitarse para agentes y carpetas de comandos confiables.

6 Proveedores, motores y despliegue

Los roles pueden reasignarse para estudios de ablación, pero el contrato de producción compacto es explícito: Codex CLI con gpt-5.6-sol y esfuerzo xhigh propone, sintetiza, revisa y analiza; AGY con gemini-3.1-pro-high y esfuerzo high co-propone, critica y navega; Claude Code con claude-opus-4-8 escribe y repara el código. La orden `scripts/audit_architecture.py` verifica este mapa, los binarios, las puertas epistémicas, los motores científicos locales y las rutas formales sin leer ni imprimir secretos. `ASTRA_REQUIRED_LOCAL_ENGINES` convierte la ausencia de un motor de producción en un fallo bloqueante de auditoría. El script idempotente `scripts/bootstrap_wsl_scientific_stack.ps1` reconstruye la pila Debian fijada. Otros proveedores compatibles se reservan para comparaciones controladas.

La instalación de escritorio crea un entorno Python y arranca la interfaz en <http://127.0.0.1:5050>. Las credenciales y parámetros del oráculo se mantienen en configuración local y no deben incorporarse al repositorio. ASTRA conserva sesiones e informes en el espacio de trabajo de la aplicación.

7 Persistencia, trazabilidad y calibración

Cada ciclo genera informes HTML y Markdown con la intuición, conjetura, script, resultado de ejecución, análisis y proveedores utilizados. Las investigaciones pueden guardarse, recargarse y revisarse desde el historial. La suite de *benchmarks* incluye problemas de relatividad general, ecuaciones diferenciales, fluidos, lógica, sistemas cuánticos y cálculo simbólico.

Los *benchmarks* cumplen dos funciones: comprobar que los motores siguen operativos y comparar la capacidad de distintos proveedores para construir pruebas discriminantes. No sustituyen la validación científica del problema nuevo, pero detectan regresiones de infraestructura y sesgos sistemáticos.

8 Garantías y límites

ASTRA mejora la reproducibilidad porque conserva y ejecuta el programa, pero no convierte automáticamente una prueba computacional en una demostración universal. Persisten varias obligaciones:

- verificar que la conjetura formal corresponde a la pregunta original;
- inspeccionar que el código no compruebe una versión debilitada o circular;
- distinguir evidencia numérica, identidad simbólica y demostración lógica;
- declarar dominios, unidades, condiciones de frontera y tolerancias;

- repetir resultados sensibles a precisión, malla o plataforma;
- someter las conclusiones relevantes a revisión humana independiente.

Por ello, **VALIDATED** significa “validado por el oráculo bajo el programa y los supuestos registrados”, no “verdad matemática absoluta”. La fortaleza de **ASTRA** reside en hacer visible y repetible la cadena de evidencia.

9 Conclusión

ASTRA Production ha evolucionado desde un esquema de procesamiento fijo hacia una arquitectura de investigación multiagente, configurable e iterativa. La formulación, la traducción, la ejecución, el análisis, la navegación y la aprobación son responsabilidades diferenciadas. El oráculo aporta una referencia externa al discurso de los modelos; los informes preservan la trazabilidad; y la puerta humana impide que la autonomía se confunda con autoridad científica. El resultado es una plataforma para investigar con modelos de lenguaje sin delegarles silenciosamente el criterio final de verdad.